

Software Copyright Source Code Submission

Statement of Particulars — Computer Programme (India)

Project Title	College Assistant — A Multi-Agent Retrieval-Augmented Question Answering System
Version	1.0
Author(s)	Dr Vikas Bhowate (Project Guide) Mr Om Gawande Mr Yash Garad Mr Harshal Ninawe Ms Aanchal Kalambe Ms Aditi Parshivanikar
Applicant / Owner	Dr Vikas Bhowate
Programming Languages	Python (Django / Django REST Framework), JavaScript (React), SQL
Frameworks & Services	Django, Django REST Framework, React, Vite, Tailwind CSS, PostgreSQL, Qdrant, Ollama (local LLM inference), Caddy
Date of Creation	2026
Nature of Work	Original literary work — Computer Programme (Source Code)

This document combines the Statement of Particulars, source-code excerpts and declaration prepared for copyright registration with the project's proprietary software licence, into a single reference file.

Software Description

College Assistant is an offline, self-hosted AI assistant that answers questions about a college's database — courses, departments, faculty, fees, schedules, and more — through a simple chat website. It combines a React/Vite web client with a Python (Django REST Framework) back-end. A multi-agent pipeline routes each question to a SQL agent for structured records, a retrieval-augmented-generation (RAG) agent for descriptive search over a vector store, and an optional web agent for official college pages; a synthesis agent then composes the final answer. All language-model inference runs locally via Ollama, so no student or institutional data leaves the server.

The software is organised into the following major modules:

Authentication & Accounts	Staff/user authentication, password hashing, JWT session issuance, and request throttling (<code>accounts/views.py</code> , <code>accounts/permissions.py</code> , <code>accounts/throttling.py</code>).
Orchestrator	Core request pipeline that routes each question across the agents below, merges their output, and applies caching and concurrency control (<code>orchestrator/service.py</code> , <code>orchestrator/views.py</code> , <code>orchestrator/cache.py</code>).
Router Agent	Classifies each incoming question via a local LLM to decide whether it needs the SQL agent, the RAG agent, or both (<code>router_agent/classifier.py</code> , <code>router_agent/llm_client.py</code>).
SQL Agent	Generates and safely executes read-only SQL against the college database behind a query guard and schema allow-list (<code>sql_agent/service.py</code> , <code>sql_agent/guard.py</code> , <code>sql_agent/schema.py</code>).
RAG Agent	Embeds each question and retrieves descriptive context from the Qdrant vector store (<code>rag_agent/service.py</code> , <code>rag_agent/vector_store.py</code> , <code>rag_agent/embedder.py</code>).
Synthesis Agent	Composes the final natural-language answer, streaming or otherwise, from the retrieved SQL/RAG/web context via a local LLM (<code>synthesis_agent/service.py</code> , <code>synthesis_agent/llm_client.py</code>).
Web & Verification Agents	Fetches and summarises official college web pages when configured, and cross-checks generated answers before they are returned (<code>web_agent/service.py</code> , <code>orchestrator/verification.py</code>).
Audit & Health	Request/action logging and health-check endpoints for each downstream dependency (<code>audit/models.py</code> , <code>health/views.py</code>).
Sync Worker	Background service that watches the college database's change queue, converts changed rows to text, generates embeddings, and upserts them into Qdrant (<code>sync_worker/</code>).
Frontend	React/Vite chat client with markdown rendering, authentication screens, sidebar, and streaming chat UI (<code>frontend/src/App.jsx</code> , <code>components/Chat.jsx</code> , <code>components/Login.jsx</code>).

System Architecture

The following diagram depicts the layered system architecture of the College Assistant platform, spanning the client, API, backend agent, external service, and data layers.

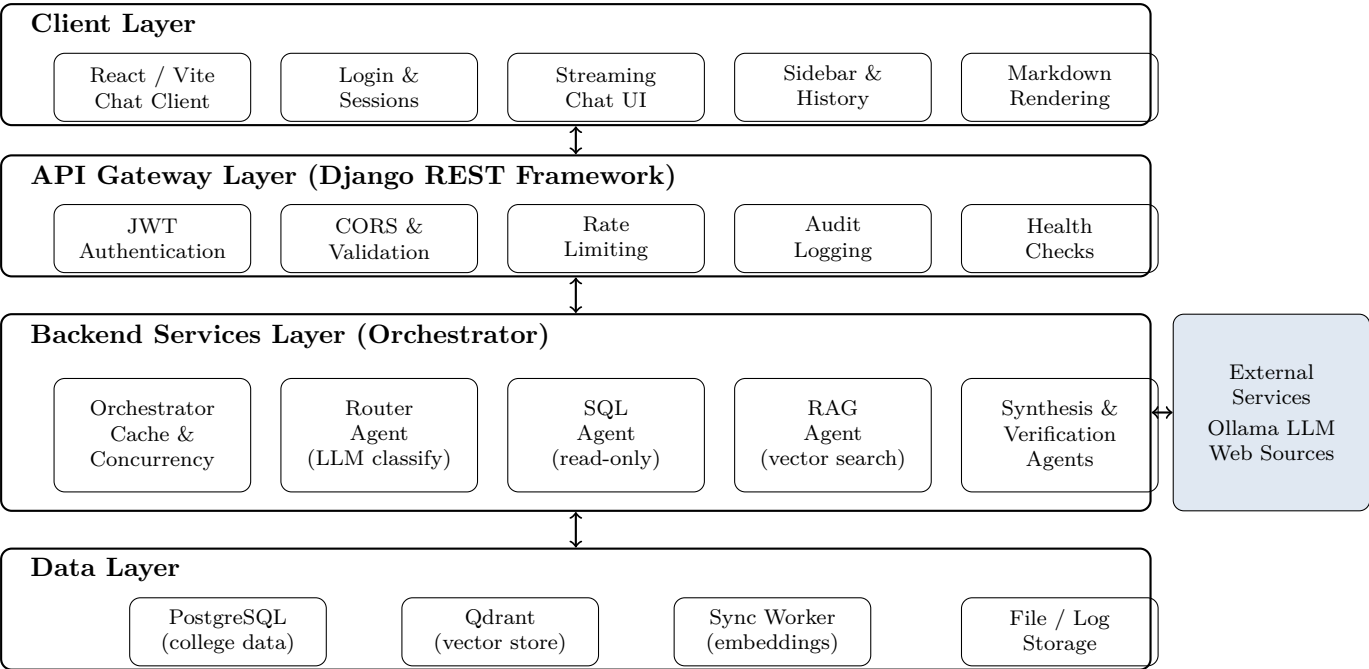


Fig. 1. System architecture of the College Assistant platform.

Source Code Inventory

#	File	Module	Language	Lines
1	manage.py	Backend / Django Entry Point	Python (Django)	21
2	config/urls.py	Backend / URL Routing	Python (Django)	20
3	config/settings/base.py	Backend / Configuration	Python (Django)	281
4	orchestrator/service.py	Backend / Multi-Agent Orchestrator	Python	395
5	router_agent/classifier.py	Backend / Query Router (LLM)	Python	87
6	sql_agent/service.py	Backend / SQL Agent	Python	90
7	rag_agent/vector_store.py	Backend / RAG Vector Store (Qdrant)	Python	106
8	synthesis_agent/service.py	Backend / Answer Synthesis (LLM)	Python	95
9	frontend/src/main.jsx	Frontend / App Entry Point	JavaScript (React)	13
10	frontend/src/App.jsx	Frontend / Root Component & Routing	JavaScript (React)	95
11	frontend/src/components/Chat.jsx	Frontend / Chat UI	JavaScript (React)	719

Note: Per the copyright office guidance, this submission reproduces the first portion and the last portion of the source code. Third-party libraries, virtual environments, container images, and build artefacts are excluded as they are not authored by the applicant.

Part A — First Portion of Source Code

Representative opening source code (multi-agent orchestrator).

Each excerpt below is reproduced verbatim from the applicant's original source code. A header identifying the file, module, and language precedes the listing.

File Name:	orchestrator/service.py	Language:	Python (Django)
Module:	Backend / Multi-Agent Orchestrator	Lines:	1–395 (complete file)

```

1  # Copyright (c) 2026 Yash Garad. All rights reserved.
2
3  import logging
4  from concurrent.futures import ThreadPoolExecutor
5
6  from common.exceptions import (
7      DatabaseUnavailable,
8      ServiceUnavailable,
9      VectorStoreUnavailable,
10 )
11 from rag_agent.embedder import embed_text
12 from rag_agent.service import retrieve
13 from router_agent.classifier import classify
14 from sql_agent.service import ask as sql_ask
15 from synthesis_agent.service import synthesize_answer, synthesize_answer_stream
16 from web_agent.service import fetch_for_question
17
18 from . import cache, verification
19 from .concurrency import llm_slot
20 from .profiling import Profile
21
22 logger = logging.getLogger("orchestrator")
23
24 DEFAULT_ROUTE_ON_ERROR = "BOTH"
25 RAG_TOP_K = 5
26
27 # User-facing degradation notes appended to an answer when one source is down
28 # but we could still answer from the other.
29 DB_DOWN_NOTE = (
30     "Note: live records lookup is temporarily unavailable, so this answer is "
31     "based on descriptive information only."
32 )
33 RAG_DOWN_NOTE = (
34     "Note: descriptive search is temporarily unavailable, so this answer is "
35     "based on the structured records only."
36 )
37 WEB_EMPTY_NOTE = (
38     "Note: no official college page matching this question could be read, so "
39     "this answer comes from the records held in the database."
40 )
41 RAG_DOWN_SQL_FALLBACK_NOTE = (
42     "Note: descriptive search is temporarily unavailable; answering from the "
43     "structured records instead."
44 )
45
46
47 def _resolve_route(question):
48     # classify() can raise LLMUnavailable (Ollama down) - we let that propagate,
49     # since if the LLM is down synthesis can't run either. A *parse* failure
50     # (route_result.error) is different: fall back to BOTH and carry on.
51     route_result = classify(question)
52     if route_result.error:
53         logger.warning(
54             "router failed question=%r error=%s - falling back to %s",
55             question, route_result.error, DEFAULT_ROUTE_ON_ERROR,
56         )
57     return DEFAULT_ROUTE_ON_ERROR, f"router error, defaulted to {DEFAULT_ROUTE_ON_ERROR}"
58     return route_result.route, route_result.reason

```

```

59
60
61 def _run_sql(question, profile):
62     # Timed INSIDE the worker thread, not around pool.submit(), so the recorded
63     # interval is when the work actually ran rather than when it was queued.
64     # That distinction is the whole point when proving the two branches overlap.
65     with profile.stage("sql"):
66         return sql_ask(question, execute=True)
67
68
69 def _run_rag(question, profile):
70     with profile.stage("rag"):
71         return retrieve(question, top_k=RAG_TOP_K)
72
73
74 def _gather_sources(question, route, profile=None):
75     """Fetch SQL and/or RAG data for the route, degrading gracefully when ONE
76     source is down. Returns (sql_result, rag_chunks, effective_route, notes).
77
78     Raises a ServiceUnavailable only when no usable source remains (e.g. a
79     SQL-only question with the database down, or a descriptive question with
80     the vector store down and no useful SQL fallback). LLMUnavailable from the
81     embed/generate steps propagates untouched - if the LLM is down, the whole
82     request can't be served anyway.
83     """
84     needs_sql = route in ("SQL", "BOTH")
85     needs_rag = route in ("RAG", "BOTH")
86     sql_result = None
87     rag_chunks = None
88     notes = []
89     profile = profile or Profile()
90
91     if route == "WEB":
92         # The web agent NEVER raises for an unreachable page: it logs the
93         # failure and returns fewer pages. So an empty list here means "no
94         # allowed page matched, or none could be read", and the honest
95         # fallback is the database rather than an error.
96         with profile.stage("web_fetch"):
97             pages = fetch_for_question(question)
98             if pages:
99                 return None, None, "WEB", notes, pages
100             logger.info("WEB route produced no pages; falling back to SQL for %r", question[:60])
101             notes.append(WEB_EMPTY_NOTE)
102             sql_result = _run_sql(question, profile)
103             return sql_result, None, "SQL", notes, []
104
105     if needs_sql and needs_rag:
106         # Independent I/O - run concurrently, but tolerate either one failing.
107         with ThreadPoolExecutor(max_workers=2) as pool:
108             f_sql = pool.submit(_run_sql, question, profile)
109             f_rag = pool.submit(_run_rag, question, profile)
110             try:
111                 sql_result = f_sql.result()
112             except DatabaseUnavailable as exc:
113                 logger.warning("SQL source down (route=BOTH), degrading to RAG-only: %s", exc)
114                 notes.append(DB_DOWN_NOTE)
115             try:
116                 rag_chunks = f_rag.result()
117             except VectorStoreUnavailable as exc:
118                 logger.warning("RAG source down (route=BOTH), degrading to SQL-only: %s", exc)
119                 notes.append(RAG_DOWN_NOTE)
120
121             if sql_result is None and rag_chunks is None:
122                 raise ServiceUnavailable() # both sources down - nothing to answer from
123             effective = "BOTH"
124             if sql_result is not None and rag_chunks is None:
125                 effective = "SQL"
126             elif rag_chunks is not None and sql_result is None:
127                 effective = "RAG"
128             return sql_result, rag_chunks, effective, notes, []
129
130     if needs_sql: # SQL-only - a DB outage here is fatal (no source to fall back on)
131         sql_result = _run_sql(question, profile)

```

```

132         return sql_result, None, "SQL", notes, []
133
134     # RAG-only
135     try:
136         rag_chunks = _run_rag(question, profile)
137     except VectorStoreUnavailable:
138         # Try SQL as a fallback; use it only if it actually found something.
139         logger.warning("RAG source down (route=RAG), attempting SQL fallback")
140         fallback = _run_sql(question, profile)
141         if fallback.rows:
142             notes.append(RAG_DOWN_SQL_FALLBACK_NOTE)
143             return fallback, None, "SQL", notes, []
144         raise # nothing useful from SQL - surface the clean "descriptive search down" message
145     return None, rag_chunks, "RAG", notes, []
146
147
148 def answer_question(question):
149     """Full pipeline, blocking. Returns a dict with the final answer plus
150     intermediate metadata. Degrades gracefully; raises ServiceUnavailable only
151     when nothing can be answered."""
152     # One slot for the whole pipeline. Raises AssistantBusy if the queue wait
153     # expires, which the API layer turns into a "busy, try again" message.
154     with llm_slot(label=f"blocking q={question[:40]!r}"):
155         route, reason = _resolve_route(question)
156         sql_result, rag_chunks, effective_route, notes, web_pages = _gather_sources(question, route)
157         answer = synthesize_answer(
158             question, effective_route, sql_result=sql_result, rag_chunks=rag_chunks,
159             web_pages=web_pages,
160         )
161         # Inside the slot: verification is more LLM work, so it must not run
162         # concurrently with someone else's answer.
163         answer, trailing, vmeta = verification.verify(
164             question, effective_route, answer,
165             sql_result=sql_result, rag_chunks=rag_chunks, web_pages=web_pages,
166         )
167     if trailing:
168         answer = answer + trailing
169     if notes:
170         answer = answer + "\n\n" + "\n".join(notes)
171
172     logger.info("answered question=%r route=%s degraded=%s", question, effective_route, bool(notes))
173     return {
174         "question": question,
175         "route": effective_route,
176         "route_reason": reason,
177         "answer": answer,
178         "degraded": bool(notes),
179         "notes": notes,
180         "sql": _sql_meta(sql_result),
181         "rag": _rag_meta(rag_chunks),
182         "web": _web_meta(web_pages),
183         "verification": vmeta,
184     }
185
186
187 def answer_question_stream(question, bypass_cache=False):
188     """Full pipeline, streaming. Yields ('meta', {...}), then ('token', str)
189     per piece, then ('done', {...}). Degradation notes (if any) are streamed
190     as trailing tokens so the user sees why the answer is limited. Raises
191     ServiceUnavailable when nothing can be answered, or AssistantBusy when no
192     LLM slot frees up in time - the API layer turns both into clean user-facing
193     messages.
194
195     NOTE ON SLOT LIFETIME: the slot is held across every `yield`, i.e. for as
196     long as the client is still reading the stream. That is intended - the model
197     is genuinely occupied for that whole period. The `finally` inside llm_slot()
198     releases it when this generator is closed, including when Django closes it
199     because the client disconnected mid-answer.
200     """
201     # REGENERATE MUST NOT BE ANSWERED FROM THE CACHE.
202     #
203     # Found while testing Prompt 27: an intermittently wrong answer ("There are
204     # 0 faculty members in the Medicine department" - the model had queried the

```

```

205 # 7-row staff directory instead of the 13,000-row survey) was stored by the
206 # semantic cache and then replayed to every subsequent user, identical every
207 # time, for the rest of the process lifetime. Verification could not catch
208 # it: the answer faithfully reported what its own (wrong) query returned.
209 #
210 # Regenerate is the one control a user has when an answer looks wrong. If it
211 # returned the same cached text it would be useless exactly when it matters,
212 # so it skips the lookup - and the fresh answer is still STORED, which
213 # overwrites the bad entry and repairs it for everyone else.
214 if bypass_cache:
215     logger.info("cache bypassed (regenerate) for %r", question[:60])
216     yield from _generate_stream(question)
217     return
218
219 # CACHE CHECK BEFORE THE SLOT, DELIBERATELY.
220 #
221 # Doing it inside llm_slot() would make every cached answer queue behind
222 # whoever is currently generating - with LLM_MAX_CONCURRENCY=1 and fifty
223 # simultaneous users, forty-nine of them would be told the assistant is busy
224 # and then be handed an answer that was sitting in memory the whole time.
225 # Out here, a hit costs no slot and blocks nobody.
226 hit = cache.lookup(question, embed_text)
227
228 # COALESCE. A plain cache does nothing for simultaneous identical questions:
229 # they all look up before the first answer exists, and all miss. If someone
230 # is already generating this exact question, wait for their result rather
231 # than queuing for a second LLM slot to compute the same thing.
232 if hit is None:
233     if cache.begin(question):
234         leader = True
235     else:
236         leader = False
237         waited = cache.await_result(question)
238         if waited and waited[0]:
239             hit = (waited[0], waited[1], "coalesced")
240         else:
241             # Leader failed or timed out - fall back to doing it ourselves.
242             leader = cache.begin(question)
243 else:
244     leader = False
245
246 if hit is not None:
247     answer, cached_route, how = hit
248     yield "meta", {
249         "question": question,
250         "route": cached_route,
251         "route_reason": "served from cache",
252         "degraded": False,
253         "cached": how,
254         "sql": None,
255         "rag": None,
256     }
257     yield "token", answer
258     yield "done", {"answer": answer, "verification": {"verification": "cached"}}
259     return
260
261 try:
262     yield from _generate_stream(question)
263 finally:
264     # Always release waiters, including on error or client disconnect.
265     # _generate_stream publishes the real result via cache.finish() on
266     # success; this is the safety net that stops followers hanging.
267     if leader:
268         cache.finish(question)
269
270
271 def _generate_stream(question):
272     """The real pipeline. Separated so the coalescing wrapper above stays
273     readable and so `finally` cleanup is unambiguous."""
274     profile = Profile()
275     with llm_slot(label=f"stream q={question[:40]!r}"):
276         profile.mark("slot_acquired")
277         with profile.stage("router"):

```



```

278         route, reason = _resolve_route(question)
279
280         # Emitted BEFORE the data stages, which are the slow part. The SPA can
281         # say "looking up records" the moment routing is decided instead of
282         # showing nothing until the first synthesis token, which on this
283         # hardware is tens of seconds later. See views.py for the SSE event.
284         yield "stage", {"stage": "routing_done", "route": route, "reason": reason}
285
286         sql_result, rag_chunks, effective_route, notes, web_pages = _gather_sources(
287             question, route, profile
288         )
289
290         yield "meta", {
291             "question": question,
292             "route": effective_route,
293             "route_reason": reason,
294             "degraded": bool(notes),
295             "sql": _sql_meta(sql_result),
296             "rag": _rag_meta(rag_chunks),
297             "web": _web_meta(web_pages),
298         }
299         yield "stage", {"stage": "sources_ready", "route": effective_route}
300
301         pieces = []
302         first_token_seen = False
303         synthesis_started = profile.total_ms
304         for piece in synthesizer.answer_stream(
305             question, effective_route, sql_result=sql_result, rag_chunks=rag_chunks,
306             web_pages=web_pages,
307         ):
308             if not first_token_seen:
309                 first_token_seen = True
310                 profile.mark("synthesis_first_token")
311                 pieces.append(piece)
312                 yield "token", piece
313         profile.stages.append({
314             "name": "synthesis",
315             "start_ms": synthesis_started,
316             "end_ms": profile.total_ms,
317             "ms": round(profile.total_ms - synthesis_started, 1),
318             "thread": "main",
319             "error": None,
320         })
321
322         # VERIFICATION runs only now, because it needs a COMPLETE answer.
323         #
324         # The user has already read the answer at this point, which is the
325         # deliberate trade-off: buffering it to verify first would add the full
326         # verification delay (~9.5s measured) BEFORE the first word appeared,
327         # and destroy streaming entirely. Instead the correction arrives a few
328         # seconds after the answer, and only when something was actually wrong.
329         streamed_answer = "".join(pieces)
330         yield "stage", {"stage": "verifying"}
331         with profile.stage("verification"):
332             _final, trailing, vmeta = verification.verify(
333                 question, effective_route, streamed_answer,
334                 sql_result=sql_result, rag_chunks=rag_chunks, web_pages=web_pages,
335             )
336         if trailing:
337             pieces.append(trailing)
338             yield "token", trailing
339
340         for note in notes:
341             note_text = "\n\n" + note
342             pieces.append(note_text)
343             yield "token", note_text
344
345         final_text = "".join(pieces)
346
347         # Cache only a clean, verified-or-unflagged answer. `degraded` covers the
348         # case where a source was down: that answer describes a temporary outage
349         # and must not be replayed once the source is back.
350         cache.store(

```

```
351         question, final_text, effective_route, embed_text, degraded=bool(notes)
352     )
353     # Hand the answer to anyone who coalesced behind this request. Done here
354     # rather than only in the wrapper's `finally` so waiters get the real
355     # text, not None.
356     if not notes:
357         cache.finish(question, final_text, effective_route)
358
359     logger.info(
360         "streamed answer question=%r route=%s degraded=%s verification=%s",
361         question, effective_route, bool(notes), vmeta.get("verification"),
362     )
363     profile.log(question)
364     yield "done", {
365         "answer": final_text,
366         "verification": vmeta,
367         # Per-stage timings ride along on the done event so latency can be
368         # measured from a real client, not only read out of server logs.
369         "profile": profile.as_dict(),
370     }
371
372
373 def _sql_meta(sql_result):
374     if sql_result is None:
375         return None
376     return {
377         "generated_sql": sql_result.generated_sql,
378         "error": sql_result.error,
379         "row_count": len(sql_result.rows) if sql_result.rows else 0,
380     }
381
382
383 def _rag_meta(rag_chunks):
384     if not rag_chunks:
385         return None
386     return [
387         {"table": c.table, "row_id": c.row_id, "score": round(c.score, 4)}
388         for c in rag_chunks
389     ]
390
391
392 def _web_meta(web_pages):
393     if not web_pages:
394         return None
395     return [p.to_dict() for p in web_pages]
```

Part B — Last Portion of Source Code

Representative closing source code (frontend entry point and root component).

Each excerpt below is reproduced verbatim from the applicant's original source code. A header identifying the file, module, and language precedes the listing.

File Name: frontend/src/main.jsx	Language: JavaScript (React)
Module: Frontend / App Entry Point	Lines: 1–13 (complete file)

```

1 // Copyright (c) 2026 Yash Garad. All rights reserved.
2
3 import React from "react";
4 import ReactDOM from "react-dom/client";
5
6 import App from "./App.jsx";
7 import "./index.css";
8
9 ReactDOM.createRoot(document.getElementById("root")).render(
10   <React.StrictMode>
11     <App />
12   </React.StrictMode>
13 );

```

File Name: frontend/src/App.jsx	Language: JavaScript (React)
Module: Frontend / Root Component & Routing	Lines: 1–95 (complete file)

```

1 // Copyright (c) 2026 Yash Garad. All rights reserved.
2
3 import { useEffect, useState } from "react";
4
5 import ChangePassword from "./components/ChangePassword";
6 import Chat from "./components/Chat";
7 import Login from "./components/Login";
8 import { ensureCsrfs, getMe, logout } from "./api";
9
10 function App() {
11   // The whole user object, not just the name: routing now depends on
12   // must_change_password as well as on being signed in.
13   const [user, setUser] = useState(null);
14   const [loading, setLoading] = useState(true);
15
16   // On load: make sure we have a CSRF cookie, then ask the server whether we
17   // already have a valid session (rather than trusting any client-side state).
18   useEffect(() => {
19     (async () => {
20       await ensureCsrfs();
21       const me = await getMe();
22       setUser(me ?? null);
23       setLoading(false);
24     })();
25   }, []);
26
27   // Paint the saved theme as soon as we know who the user is, so the login and
28   // change-password screens are already in the right mode rather than flashing
29   // the wrong one.
30   //
31   // SIGNED OUT, the profile does not exist yet, so the sign-in screen's own
32   // preference is honoured instead (Login.jsx keeps it in localStorage - the
33   // only store available before there is an account). This used to force dark
34   // for every signed-out visitor, which meant choosing light on the sign-in
35   // screen was undone on the next page load.
36   //

```

```
37 // SIGNED IN, the profile wins unconditionally. That ordering is the point:
38 // the account's preference follows the person between machines, and the local
39 // value is only ever a first-paint fallback.
40 useEffect(() => {
41   let dark;
42   if (user) {
43     dark = user.theme !== "light";
44   } else {
45     let saved = null;
46     try {
47       saved = localStorage.getItem("theme");
48     } catch { /* private browsing can throw */ }
49     dark = saved ? saved === "dark" : true;
50   }
51   document.documentElement.classList.toggle("dark", dark);
52 }, [user]);
53
54 async function handleLogout() {
55   await logout();
56   setUser(null);
57 }
58
59 function handleSessionExpired() {
60   // Session was rejected mid-use - bounce back to login.
61   setUser(null);
62 }
63
64 if (loading) {
65   return (
66     <div className="flex min-h-screen items-center justify-center bg-white text-neutral-400 dark:bg-[#0
67       f0f10] dark:text-neutral-500">
68       ..Loading
69     </div>
70   );
71 }
72
73 if (!user) {
74   return <Login onLoggedIn={setUser} />;
75 }
76
77 // Checked BEFORE rendering Chat. The server enforces this too - /api/ask/
78 // returns 403 while the flag is set - but showing a chat box that rejects
79 // every question would be a confusing way to communicate that.
80 if (user.must_change_password) {
81   return (
82     <ChangePassword user={user} onChanged={setUser} onLogout={handleLogout} />
83   );
84 }
85
86 return (
87   <Chat
88     username={user.username}
89     initialTheme={user.theme}
90     onLogout={handleLogout}
91     onSessionExpired={handleSessionExpired}
92   />
93 );
94 }
95 export default App;
```

Declaration

I/We hereby declare that the source code excerpts submitted herewith form part of the computer programme titled “**College Assistant — A Multi-Agent Retrieval-Augmented Question Answering System**”, that the said work is an original work created by the applicant(s), and that the applicant(s) are the rightful owner(s) of the copyright therein. The source code is submitted solely for the purpose of registration of copyright and is true and correct to the best of my/our knowledge and belief.

Signature of Applicant	Date
Dr Vikas Bhowate (Project Guide)	
Mr Om Gawande	
Mr Yash Garad	
Mr Harshal Ninawe	
Ms Aanchal Kalambe	
Ms Aditi Parshivanikar	

Place: _____

PROPRIETARY SOFTWARE LICENSE

All Rights Reserved

College Assistant

A Multi-Agent Retrieval-Augmented Question Answering System

Copyright © 2026 Dr Vikas Bhowate. All rights reserved.

1. Ownership

This software, including all source code, object code, configuration files, database schemas, prompt designs, documentation, and any accompanying materials (collectively, the “Software”), is the exclusive property of Dr Vikas Bhowate (the “Owner”) and is protected by the Copyright Act, 1957 (India), as amended, and by applicable international copyright treaties and conventions. The academic project team named in the Statement of Particulars and Declaration above — Dr Vikas Bhowate, Mr Om Gawande, Mr Yash Garad, Mr Harshal Ninawe, Ms Aanchal Kalambe, and Ms Aditi Parshivanikar — are credited as authors of the work; this licence governs the terms on which the Software may be used by any party other than the Owner.

The Software is licensed, not sold. No transfer of ownership, title, or intellectual property rights of any kind is made or implied by the possession, receipt, or use of a copy of the Software.

2. No License Granted

No rights or licenses are granted under this document, whether by implication, estoppel, or otherwise, except as expressly stated in a separate written agreement signed by the Owner.

Without the prior express written permission of the Owner, no person or entity may:

- (a) copy, reproduce, or duplicate the Software, in whole or in part;
- (b) modify, adapt, translate, or create derivative works from the Software;
- (c) distribute, publish, sublicense, sell, rent, lease, lend, or otherwise transfer the Software or any copy thereof to any third party;
- (d) host, deploy, or make the Software available as a service to any third party, whether or not for a fee;
- (e) reverse engineer, decompile, or disassemble the Software, except to the limited extent such restriction is expressly prohibited by applicable law;
- (f) remove, obscure, or alter any copyright notice, proprietary legend, or attribution contained in or on the Software; or
- (g) use the Software for any commercial purpose.

3. Authorized Use

Any permitted use of the Software is governed solely by the terms of a separate written licence agreement executed by the Owner. In the absence of such an agreement, no use is authorized.

4. Third-Party Components

The Software is designed to operate together with third-party software components that are **not** owned by the Owner and are **not** covered by this licence. Each such component remains subject to its own licence terms, and nothing in this document modifies, supersedes, or restricts those terms.

These components are obtained at build time via the declared dependency manifests and are not distributed as part of the Software. They include, without limitation:

Backend (*see `backend/requirements.txt`*)

Django, Django REST Framework, django-cors-headers, psycpg2-binary, python-dotenv, gunicorn, sqlglot, qdrant-client, requests

Frontend (*see `frontend/package.json`*)

React, React DOM, Vite, @vitejs/plugin-react, Tailwind CSS, PostCSS, Autoprefixer

Container images (*see `docker-compose.yml`*)

PostgreSQL, Qdrant, Ollama, Caddy, and the language models served by Ollama

The Owner's claim of copyright extends only to the original work of authorship in the Software – the application source code, architecture, agent designs, prompt engineering, security controls, and documentation authored by the Owner and the project team – and does not extend to any third-party component listed above.

5. Disclaimer of Warranty

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, ACCURACY, AND NON-INFRINGEMENT.

The Software incorporates automated language-model-generated output. Such output may be incomplete, inaccurate, or otherwise unsuitable for any particular purpose, and must not be relied upon as authoritative without independent verification.

6. Limitation of Liability

IN NO EVENT SHALL THE OWNER BE LIABLE FOR ANY CLAIM, DAMAGES, OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT, OR OTHERWISE, ARISING FROM, OUT OF, OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

7. Governing Law

This licence shall be governed by and construed in accordance with the laws of India, without regard to its conflict-of-law provisions. The courts of India shall have exclusive jurisdiction over any dispute arising out of or in connection with this licence.

8. Contact

For licensing enquiries, permissions, or any other matter relating to this Software, contact the Owner:

Name: Dr Vikas Bhowate

Email: vipbhowate@gmail.com

END OF TERMS